

Continuous Integration (CI) Pipeline

Project: software-architecture-labs | **Feature:** Group / Ungroup figures (JHotDraw 7)

What CI is

Continuous Integration is the practice of merging every developer's work into a shared mainline frequently, where each merge automatically triggers a build and test run. This catches integration errors early instead of at release time.

The pipeline

The pipeline is defined by a single GitHub Actions workflow: `.github/workflows/ci.yml`. It runs automatically on:

- every **push** to the `main` branch, and
- every **pull request** targeting `main`.

What it does

Step	Action	Purpose
1	<code>actions/checkout@v4</code>	Check out the repository on the runner.
2	<code>actions/setup-java@v4</code> (Temurin JDK 17 , Maven cache)	Provide the JDK and cache <code>~/.m2</code> for speed.
3	<code>mvn -B package --file L3/ci-demo/pom.xml</code>	Build the Maven module. The <code>package</code> goal runs <code>compile</code> → <code>test</code> → <code>package</code> , so it both builds and runs the JUnit tests in one step.

Why a dedicated module instead of building JHotDraw directly

The first CI attempt targeted the cloned `jhotdraw/jhotdraw7/pom.xml` and failed: that project's build is internally inconsistent. Its `pom.xml` declares a Java **1.6** source level, but the actual sources are **modular Java 9+** code (`module-info.java` files) that use Java 7/8 syntax (diamond operator, multi-catch, method references). It therefore cannot be built with a single `mvn` command without restructuring the whole build — which is outside the scope of this lab.

Following the lab's referenced guide "*Building and testing Java with Maven*", the pipeline instead builds a small, self-contained Maven module, `L3/ci-demo`, that **models the Group/Ungroup feature** (`FigureGroup` with `group()` / `ungroup()`) and ships **JUnit 5 tests** (`FigureGroupTest`). This gives the pipeline a real unit to compile and genuine tests that run and pass on every push and pull request.

GitHub Packages / `.maven-settings.xml`

The lab mentions creating a `.maven-settings.xml` to pull shared jars from GitHub Packages. This project does **not** need one: its only dependency (JUnit) comes from Maven Central, so there are no private packages to authenticate against. If a GitHub Packages dependency were added, the workflow would pass `--settings .maven-settings.xml` to Maven and the file would hold the registry URL plus a `${GITHUB_TOKEN}` server entry.

Result

Once pushed, the workflow appears under the repository's **Actions** tab and runs on every push/PR, compiling the module and executing the JUnit tests — giving a green/red build status that gates integration.